Introduction to Operating Systems and Processes

An operating system (OS) is the software layer that manages hardware resources and provides services to application programs. A process is a running instance of a program; it has its own address space, a program counter, registers, and a set of open file descriptors. The OS is responsible for creating, scheduling, and terminating processes, as well as isolating one process's memory from another's.

Each process moves through states during its lifetime: new, ready, running, waiting (blocked), and terminated. A process in the ready state is waiting for the CPU scheduler to give it a time slice; a process in the waiting state is blocked on an I/O operation or another event and cannot run even if given the CPU.

Process Scheduling: FCFS and SJF

First-Come, First-Served (FCFS) scheduling runs processes strictly in the order they arrive in the ready queue. FCFS is simple to implement but can cause the "convoy effect," where a long process at the front of the queue makes many short processes behind it wait, increasing average waiting time.

Shortest Job First (SJF) scheduling instead picks the ready process with the smallest estimated run time next. SJF minimizes average waiting time among non-preemptive scheduling algorithms when job lengths are known in advance, but it requires predicting run time and can starve long jobs if short jobs keep arriving.

Process Scheduling: Round Robin and Priority Scheduling

Round Robin (RR) scheduling gives each ready process a fixed time slice, called a quantum, and cycles through the ready queue. RR is preemptive: if a process does not finish within its quantum, it is moved to the back of the queue. A small quantum improves responsiveness but increases context-switch overhead; a large quantum makes RR behave more like FCFS.

Priority scheduling assigns each process a priority number and always runs the highest-priority ready process. Without safeguards, priority scheduling can starve low-priority processes indefinitely; a common fix is "aging," which gradually increases the priority of processes that have waited a long time.

Deadlocks: Conditions and Prevention

A deadlock occurs when a set of processes are each waiting for a resource held by another process in the set, so none of them can proceed. Four conditions must all hold simultaneously for a deadlock to occur: mutual exclusion, hold-and-wait, no preemption, and circular wait.

Deadlocks can be prevented by denying at least one of these four conditions. For example, requiring processes to request all resources at once denies hold-and-wait, and imposing a strict global ordering on resource acquisition denies circular wait. Deadlock avoidance algorithms, such as the Banker's Algorithm, instead grant requests dynamically only if the resulting state is still safe.

Virtual Memory and Paging

Virtual memory lets each process address more memory than is physically installed, and gives each process the illusion of a private, contiguous address space. The OS divides virtual addresses into fixed-size pages and physical memory into same-sized frames, maintaining a page table that maps pages to frames.

When a process accesses a page that is not currently in a physical frame, a page fault occurs and the OS loads the page from disk (the backing store). If physical memory is full, the OS must evict a page using a replacement policy such as Least Recently Used (LRU) or the clock algorithm, which approximates LRU more cheaply.

Segmentation and File Systems

Segmentation divides a process's address space into logically meaningful variable sized units, such as code, stack, and heap segments, rather than fixed-size pages. Segmentation can be combined with paging, giving each segment its own page table.

A file system organizes persistent storage into files and directories and tracks, for each file, where its data blocks live on disk. Most file systems use an index structure, such as an inode, to record a file's metadata and the block pointers needed to read its contents, and directories map human-readable names to these index structures.